✈ AIROPLANE TICKET SERVICE

A Flight Reservation System with a Native Desktop Application, a Web Interface and an Android Client

Project Title	Airoplane Ticket Service — Flight Reservation & E-Ticket System
Student Name	_____
Roll Number	_____
Class / Section	_____
Guide / Faculty	_____
Department	Computer Science & Engineering
College / Institution	_____
Academic Year	2025 – 2026
Date of Submission	_____

Course: Software Engineering / Full-Stack Development Project
Languages used: Java 17, HTML5, CSS3, JavaScript (ES6), JSON, Shell
Technologies: JDK `HttpServer`, Java Swing, Maven, Node.js + Capacitor, Android Gradle, `jpackage`

Abstract

Online flight reservation systems are widely used, but most of them hide the most important part of the process: a booking is not a ticket. In a real airline, the customer searches a route, selects a seat, enters passenger details and pays, but the e-ticket is issued only after the booking has been checked and approved
by the airline. The **Airoplane Ticket Service** is a flight-ticket booking system built around exactly this
rule.

The system is developed in **pure Java (version 17) without any third-party libraries**, and it is delivered
in three forms from a single source tree: a **native desktop application** built with Java Swing, a **responsive web / installable PWA**, and an **Android application** built with Capacitor. The backend uses
the HTTP server included in the JDK, a hand-written JSON parser and serialiser, SHA-256 password hashing
with token-based sessions, and file-based JSON storage. The code is organised into four layers — model, service, store and user interface — so that the business rules are written once and reused by every front end.

A customer registers or logs in, searches flights by source airport, destination airport, date and number of passengers, and receives a result list showing price, schedule, duration and live seat availability. A 30 × 6 interactive seat map (180 seats) is displayed, where already-booked seats cannot be selected and the
number of selectable seats is limited to the number of passengers. Passenger details (name, age, gender) are
collected and validated for every traveller, payment is handled by a simulated gateway that validates the card number and issues a transaction reference, and the total price is recalculated live.

Every new booking is created with the status **PENDING**. The seat is reserved so that no other customer can
take it, but no valid e-ticket is issued and printing remains locked. The administrator reviews pending bookings in a separate dashboard, confirms or cancels them, and only on confirmation does the booking become
CONFIRMED and the e-ticket printable. Revenue statistics count confirmed bookings only. An airports
module with a custom-drawn, animated map of India and the domestic route network allows the customer to
start a search directly from a city.

The system was verified with a ten-step service-level test of the approval workflow, live HTTP tests of every endpoint including role enforcement, and automated pixel-level rendering tests of the graphical user
interface.

Keywords: Java, Swing, HTTP server, JSON, authentication, SHA-256, seat booking, admin approval workflow, PWA, Capacitor, file-based persistence.

1. Objectives

1. To build a complete flight-search, seat-booking, payment and e-ticket system with no external dependencies, so that it runs on any machine with JDK 17.
2. To model the real airline approval workflow, where a booking is pending until an administrator confirms
   it and the e-ticket is issued.
3. To deliver the same application as a native desktop app, a web/PWA and an Android app using a single
   shared business-logic layer.
4. To persist data without a database server, using a thread-safe file-based JSON store.
5. To provide a secure-enough authentication mechanism with hashed passwords and bearer-token sessions.

2. Languages and technologies used

Language / technology	Purpose
Java 17	Backend API, business logic, data persistence and the complete desktop GUI (Swing) — 31 source files, ~3,700 lines
HTML5 / CSS3	Web / PWA layout, forms, seat-map grid, modals and dashboard styling
JavaScript (ES6)	Front-end API client, search, booking flow, admin dashboard, service worker
JSON	Data-storage format and the body format of every API request and response
Java Swing / AWT	Native desktop application: main window, tabs, dialogs, custom painting, printing
Service Worker / Web App Manifest	Making the web app installable and usable offline on phones
Node.js + Capacitor 7	Building the Android application from the same web interface
Maven / Gradle	Java and Android build configuration
Shell (Bash)	One-command compile-and-run script, double-click launcher and APK build script
JDK `jpackage` + `codesign`	Producing the signed macOS `.app` bundle

3. Methodology

The system was developed using the **waterfall model with iterative prototyping**. The first iteration
implemented the core backend — the HTTP server, routing, authentication and the flight search — together
with the JSON store. The second iteration added the booking workflow, seat allocation, mock payment and the
e-ticket. The third iteration added the administration dashboard and the flight management module, and the
final iteration produced the Swing desktop front end, packaged the application for macOS and Android, and
carried out the testing described in section 6.

The architecture follows a **layered design**, in which each layer has a single responsibility:

Layer	Package	Responsibility
Model	`com.airline.model`	`Flight`, `User` and `Booking` objects and their JSON conversion
Service	`com.airline.service`	All business rules: authentication, flight search, booking, approval, cancellation
Store	`com.airline.store`	File-based JSON persistence, seeding of demo data, date rolling
Utility	`com.airline.util`	Hand-written JSON parser, HTTP helpers, SHA-256 password hashing
UI	`com.airline.ui`	Native Swing desktop application with a custom neumorphic (soft-UI) theme (20 classes)

The **desktop application calls the service layer directly within the same JVM process**, so it does not
need to serialise data to JSON or send it over a socket. The web and Android applications use the HTTP API
instead. Because both paths end at the same service methods, a rule such as "a booking stays pending until
an administrator confirms it" is enforced identically in all three applications.

---

4. System modules

Module	Description
Authentication	Registration and login, SHA-256 hashed passwords, random UUID bearer tokens, in-memory session store, `customer` and `admin` roles
Flight search	Filter by source, destination and date; shows schedule, duration, price and remaining seats; seeded with Indian domestic routes (DEL, BOM, BLR, HYD, MAA)
Seat selection	Interactive 30 × 6 seat map with an aisle, live taken/free states, seat holds and a re-check at booking time to prevent double booking
Passenger details	Name, age and gender per traveller, with labelled columns, placeholder hints and age validation
Mock payment	Card-number validation, simulated `PAY-XXXXXXXX` transaction and a mocked refund on cancellation
Approval workflow	Booking states `PENDING → CONFIRMED / CANCELLED`, admin-only confirmation, `confirmedAt` audit stamp, revenue counted from confirmed bookings only
E-ticket	PNR, flight details, passenger table, seat numbers, total amount; printable from the desktop app and the browser
My bookings	Booking history with status badges, pending notice, approval check, and cancellation that releases the seat
Admin dashboard	Statistics (flights, users, bookings, active, pending, revenue), flight add/edit/delete, and the pending-approval queue
Airports module	Map of the served cities: a custom-painted, animated map of India in the desktop app, and a Leaflet + OpenStreetMap map in the web and mobile apps, with the airport list and a "fly from here" shortcut
Desktop application	Native Swing window packaged as a signed macOS `.app` bundle using `jpackage` — no browser required
Web / PWA and Android	The same interface as an installable web app and as a Capacitor-based Android APK

5. Main code

5.1 Creating a booking — status `PENDING`

`src/main/java/com/airline/service/BookingService.java`

```
public Result create(String userId, String flightId, List<Map<String, Object>> passengers,
                     List<String> seats, String paymentId, double paidAmount) {
    Result r = new Result();
    synchronized (db) {
        Flight f = db.flights.get(flightId);
        if (f == null) { r.error = "Flight not found."; return r; }
        if (passengers == null || passengers.isEmpty()) { r.error = "Add at least one passenger.";
return r; }
        if (seats == null || seats.size() != passengers.size()) { r.error = "Seats must match
passengers."; return r; }
        for (String s : seats) {
            if (f.bookedSeats.contains(s)) { r.error = "Seat " + s + " just got booked. Pick
another."; return r; }
        }
        double total = f.price * passengers.size();
        Booking b = new Booking();
        b.pnr = genPnr();
        b.userId = userId; b.flightId = flightId;
        b.passengers = passengers; b.seats = new ArrayList<>(seats);
        b.totalPrice = total;
        b.status = "PENDING";          // the admin must confirm before the ticket is issued
        b.paymentId = paymentId == null ? "" : paymentId;
        b.createdAt = LocalDateTime.now().toString();
        f.bookedSeats.addAll(seats);   // the seat is held while the booking waits
        db.bookings.put(b.pnr, b);
        db.saveQuietly();
        r.booking = b;
    }
    return r;
}
```

5.2 Administrator confirmation — issuing the ticket

`src/main/java/com/airline/service/BookingService.java`

```
/** Admin approves a pending booking and issues the ticket. */
public Result confirm(String pnr, boolean isAdmin) {
    Result r = new Result();
    synchronized (db) {
        Booking b = db.bookings.get(pnr);
        if (b == null) { r.error = "Booking not found."; return r; }
        if (!isAdmin) { r.error = "Only an admin can confirm a booking."; return r; }
        if (b.status.equals("CONFIRMED")) { r.error = "Booking " + pnr + " is already confirmed.";
return r; }
        if (b.status.equals("CANCELLED")) { r.error = "Booking " + pnr + " was cancelled."; return
r; }
        b.status = "CONFIRMED";
        b.confirmedAt = LocalDateTime.now().toString();
        db.saveQuietly();
        r.booking = b;
    }
    return r;
}
```

5.3 API routing and role enforcement

`src/main/java/com/airline/App.java`

```
// Create a booking — the customer is authenticated, the result is PENDING
if (path.equals("/api/bookings") && method.equals("POST")) {
    User u = requireLogin(ex); if (u == null) return;
    Map<String, Object> b = Json.parseObject(HttpUtil.readBody(ex));
    BookingService.Result r = bookings.create(u.id, Json.str(b, "flightId", ""), passengers,
seats,
            Json.str(b, "paymentId", ""), Json.dbl(b, "paidAmount", 0));
    if (r.error != null) { json(ex, 400, err(r.error)); return; }
    json(ex, 201, ok(Map.of("booking", enrich(r.booking)))); return;
}

// Approve a pending booking — administrators only
if (path.startsWith("/api/bookings/") && path.endsWith("/confirm") && method.equals("POST")) {
    User u = requireAdmin(ex); if (u == null) return;
    String pnr = path.substring("/api/bookings/".length(), path.length() - "/confirm".length());
    BookingService.Result r = bookings.confirm(pnr, true);
    if (r.error != null) { json(ex, 400, err(r.error)); return; }
    json(ex, 200, ok(Map.of("booking", enrich(r.booking)))); return;
}

// Only an administrator may reach the approval endpoint
private User requireAdmin(HttpExchange ex) throws IOException {
    User u = auth.me(HttpUtil.bearer(ex));
    if (u == null) { json(ex, 401, err("Login required.")); return null; }
    if (!u.role.equals("admin")) { json(ex, 403, err("Admin only.")); return null; }
    return u;
}
```

5.4 Password hashing and token sessions

`src/main/java/com/airline/util/Passwords.java` and `com/airline/service/AuthService.java`

```
// SHA-256 hash — no plain-text password is ever written to disk
public static String hash(String plain) {
    MessageDigest md = MessageDigest.getInstance("SHA-256");
    byte[] d = md.digest(plain.getBytes(StandardCharsets.UTF_8));
    StringBuilder sb = new StringBuilder();
    for (byte b : d) sb.append(String.format("%02x", b));
    return sb.toString();
}

// Login — verifies the hash and issues a random session token
public AuthResult login(String email, String password) {
    AuthResult r = new AuthResult();
    User u = db.findUserByEmail(email == null ? "" : email.trim());
    if (u == null || !Passwords.verify(password == null ? "" : password, u.passwordHash)) {
        r.error = "Invalid email or password."; return r;
    }
    r.user = u; r.token = issueToken(u);
    return r;
}

private String issueToken(User u) {
    String t = UUID.randomUUID().toString();
    db.sessions.put(t, u.id);   // token → userId
    return t;
}
```

5.5 File-based storage and automatic seeding

`src/main/java/com/airline/store/DataStore.java`

```
public synchronized void load() throws IOException {
    Files.createDirectories(dataDir);
    loadFlights(); loadUsers(); loadBookings();
    if (flights.isEmpty()) seedFlights();    // ten seeded Indian domestic flights
    if (users.isEmpty()) seedUsers();        // admin@skyjet.in and demo@skyjet.in
    rollFlightDatesForward();                // keeps the demo dates in the future
    saveAll();
}
```

6. Testing

Test	Result
Service-level test of the complete workflow: create → `PENDING`, customer confirmation rejected, administrator confirmation → `CONFIRMED`, duplicate confirmation rejected, cancellation of a pending booking releases the seat	Pass
Live HTTP test of all endpoints, including that a customer receives HTTP 403 when attempting to confirm a booking and that the pending count is reported in the statistics	Pass
User-interface test confirming the pending badge, the disabled print button and the administrator's confirm action	Pass
Pixel-level rendering test of the screens, used to detect and correct a defect in which input fields were drawn white on white and therefore invisible	Pass
Clean compilation of all 24 Java source files and successful launch of the packaged macOS application	Pass

7. Conclusion

The project delivers a working flight-booking system that covers the complete customer journey — registration, flight search, seat selection, passenger details, payment, approval and e-ticket — and makes it
available as a native desktop application, a web application and an Android application from a single code
base. Its defining feature is the **admin approval gate**: the customer pays and reserves a seat, but the e-ticket is issued only after an authorised administrator confirms the booking, and every state transition is validated, persisted and clearly displayed in the user interface.

Building the system without third-party libraries proved the value of the layered design. The desktop application, the browser and the phone all reuse one service layer, so a business rule is implemented and
tested exactly once. The project also demonstrates that a project can remain completely self-contained and

run on any machine with a JDK installed, while still offering a modern, animated and fully functional user
interface.

---

8. Limitations and future scope

Limitations: password hashing is unsalted SHA-256 and should be replaced by bcrypt or Argon2 in production; storage is file-based, so the system runs as a single process and does not scale to many concurrent users; the payment gateway is simulated; the desktop map is drawn by the program itself and
therefore has no live tile layer, and the web map requires an internet connection to load its tiles; and the web interface loads Leaflet from a public content-delivery network, so it would need to be bundled locally for a fully offline installation.

Future scope: integration with a real payment gateway, seat-hold expiry timers, e-mail and SMS ticket
delivery with a QR code, PDF e-ticket generation, support for multiple airlines with fare classes and dynamic pricing, live notifications when an administrator approves a booking, and migration of the file store to SQLite or PostgreSQL.

Repository: `https://github.com/Mrkarthik123/airoplane-ticket-service` *(branch* `main`*)*
Demo accounts — customer: `demo@skyjet.in` / `demo123`*, administrator:* `admin@skyjet.in` / `admin123`